

Aufgabe 3: Lieferkette

Teilnahme-ID: 81342

Bearbeiter: Raphael Porsche

Inhaltsverzeichnis

1. Lösungsidee
 1. Problemdefinition
 2. Dynamische Programmierung mit drei Gewichtungen
 3. Korrektheitsbeweis des Ansatzes
 4. Laufzeitanalyse
 5. Cache-Lokalität und Speicheroptimierung
 6. Potentielle Optimierung
 7. Datenstrukturen
 2. Umsetzung
 1. Dijkstra-Algorithmus mit früher Terminierung
 2. Minimierungsphase und Pfadrekonstruktion
 3. Beispiele
 4. Quellcode
-

1 Lösungsidee

1.1 Problemdefinition

Gegeben sei ein ungerichteter Graph $G = (V, E)$. V setzt sich aus dem Startpunkt $S \in \mathbb{R}^2$, dem Endpunkt $T \in \mathbb{R}^2$ und eine Menge von Werken $W = \{w_1, w_2, \dots\}$ zusammen. Jedes Werk $w \in \mathbb{R}^2$ gehört zu genau einem von m verschiedenen Unternehmen $\text{firm}(w) \in \{1, 2, \dots, m\}$. So gibt es auch für jedes Unternehmen i eine Menge von Werken $F_i = \{w \in W : \text{firm}(w) = i\}$. Zwischen den Punkten in V gibt es eine Menge von gewichteten Kanten E mit einem Gewicht > 0 . Die Aufgabe liegt darin, Wege durch G zu finden, die folgende Bedingungen einhalten:

- Der Weg started bei S und endet bei T .
- Mindestens jeweils ein Werk von allen Unternehmen muss in aufsteigender Reihenfolge angefahren und der Arbeitsschritt dort ausgeführt werden.
- Bis zu maximal ein $w_j \in W$ kann streiken. Dies bedeutet, dass es unmöglich wird den Arbeitsschritt $\text{firm}(w_j)$ dort auszuführen.
- Wir erfahren von dem Streik erst, wenn wir dort angekommen sind und müssen dann ein Ersatzweg parat haben.

Ziel: Wir müssen einen Weg und mehrere Ersatzwege für jedes möglicherweise streikende w in G finden. Diese Wege müssen alle im schlechtesten Fall eine gesamt Länge $\leq$ einen vorgegebenen Wert $L_{\max}$ haben. Ist dies nicht möglich, müssen wir das zurückgeben.

1.2 Dynamische Programmierung mit drei Gewichtungen

Um das “Worst-Case”-Szenario zu ermitteln, verwenden wir ein dynamisches Programmierungsverfahren. Wir beginnen bei T und gehen rückwärts durch die Unternehmen. Jedem Punkt in V werden drei verschiedene Gewichtungen zugewiesen. Diese Gewichtungen erfassen die minimalen Kosten in den drei möglichen Modi bezüglich eines Streiks. Die Kosten beschreiben hier die minimale Weglänge um alle weiteren Firmen anzufahren und an T anzukommen.

Definition der drei Modi:

1. **modes[0] (Streik schon passiert):** Die minimalen Kosten, wenn bereits ein Streik stattgefunden hat. Dies ist einfach die kürzeste Distanz zu einem Werk des nächsten Unternehmens plus dessen “Streik schon passiert”-Kosten. Wir wählen das Werk des nächsten Unternehmen das diesen Wert minimiert.

$$\text{modes}[0](w) = \min_{u \in F_{i+1}} (\text{dist}(w, u) + \text{modes}[0](u))$$

2. **modes[2] (Streik passiert jetzt):** Die minimalen Kosten, wenn genau dieses Werk bestreikt wird. Dies ist einfach die kürzeste Distanz zu einem anderen Werk des gleichen Unternehmens plus dessen “Streik schon passiert”-Kosten. Wir wählen das Werk des gleichen Unternehmen das diesen Wert minimiert. Hierbei dürfen wir natürlich das streikende Werk nicht mit einbeziehen, da es nicht den Arbeitsschritt ausführen kann.

$$\text{modes}[2](w) = \min_{u \in F_i \setminus \{w\}} (\text{dist}(w, u) + \text{modes}[0](u))$$

3. **modes[1] (Streik wird noch passieren):** Die minimalen Kosten, wenn noch kein Streik stattgefunden hat, aber noch ein Streik auftreten kann. Dies ist die kürzeste Distanz zu einem Werk des nächsten Unternehmens plus das Maximum aus dessen “Streik wird noch passieren” und “Streik passiert jetzt”-Kosten.

$$\text{modes}[1](w) = \min_{u \in F_{i+1}} (\text{dist}(w, u) + \max(\text{modes}[1](u), \text{modes}[2](u)))$$

Interpretation: Durch diese drei Modi können wir den Worst-Case exakt modellieren. **modes[1]** repräsentiert den “worst case”, bei dem wir annehmen, dass der Gegner (der Streik) die für uns schlechteste Entscheidung trifft. Durch die Minimierung des schlechtesten Falls über alle möglichen Pfade erreichen wir eine optimale Lösung. Um unsere Lösung zu bekommen müssen wir nur an dem Punkt S **modes[1]** ablesen.

Sonderfall: Hat eine Firma nur ein einziges Werk, wird das Problem unmöglich, da ein möglicher Streik an diesem Werk uns nicht mehr erlaubt den jeweiligen Arbeitsschritt durchzuführen. Da unser Gegner (der Streik) immer die für uns schlechteste Option wählt, können wir in keinem Fall garantieren vor dem Zeitlimit $L_{\max}$ anzukommen.

1.3 Korrektheitsbeweis des Ansatzes

Wir beweisen die Korrektheit des DP-Ansatzes durch Rückwärtsinduktion über die Unternehmen von m (letztes Unternehmen vor dem Ziel) bis 1 (Start). Wir zeigen, dass die Zustände **modes[0]**, **modes[1]** und **modes[2]** für jedes Werk $w \in F_i$ die minimalen Kosten unter Worst-Case-Bedingungen korrekt berechnen. Das Problem lässt sich als ein Spiel gegen einen Gegner (den Streik) betrachten. Wir versuchen, die Weglänge zu minimieren, während der Gegner versucht, durch Wahl des Streikortes (oder Verzicht auf einen Streik) unsere Weglänge zu maximieren. Jeder “Zug” bildet hierbei eine Firma ab, wo wir uns für ein Werk entscheiden müssen, dass wir anpeilen während der Gegner entscheidet, ob er seinen einmaligen Streik einsetzt.

Induktionsanfang ($i = m$): Für ein Werk $w \in F_m$ berechnen wir die initialen Kosten zum Ziel T . Das Ziel T selbst ist kein Werk und kann laut Aufgabenstellung nicht bestreikt werden.

- **modes[0] (w)** (Streik bereits verbraucht): Da kein Streik mehr eintreten kann, sind die Kosten exakt die Distanz zum Ziel: $\text{dist}(w, T)$.

- **modes[1](w)** (Streik noch möglich): Wir befinden uns in w und der Streik hat hier nicht stattgefunden (sonst wären wir in modes[2]). Da wir nun direkt zum Ziel T aufbrechen und T nicht streiken kann, verfällt die Möglichkeit des Gegners, noch einen Streik auszulösen. Folglich gilt hier das Gleiche wie bei modes[0]: Die Kosten sind exakt $\text{dist}(w, T)$.
- **modes[2](w)** (Streik in w): Wir erreichen w und es wird gestreikt. Wir müssen zu einem Ersatzwerk $u \in F_m$ ausweichen. Da der Streik nun verbraucht ist, können wir von u direkt zum Ziel T fahren (was modes[0](u) entspricht). Die Kosten betragen korrekt: $\min_{u \in F_m \setminus \{w\}} (\text{dist}(w, u) + \text{dist}(u, T))$.

Induktionsschritt ($i \Rightarrow i - 1$): Wir nehmen an, dass die drei Modi für alle Werke $u \in F_i$ die optimalen Kosten bis zum Ziel T korrekt angeben. Wir beweisen nun die Korrektheit für ein Werk $w \in F_{i-1}$.

Beweis für modes[0] (Streik ist bereits verbraucht): Da die Spielregel besagt, dass maximal ein Werk streiken darf, kann auf dem restlichen Weg kein Streik mehr stattfinden, wenn bereits einer passiert ist. Das Worst-Case-Problem reduziert sich dazu, den kürzesten Pfad zu finden. So ist das Minimum der Distanzen zu den Nachfolgern $u \in F_i$ plus deren restliche Distanz zum Ziel (modes[0](u)) die korrekten minimalen Kosten um die restlichen Firmen und T zu erreichen.

Beweis für modes[2] (Streik trifft genau Werk w): Wir erreichen w und erfahren von einem Streik an der jetzigen Position. Da wir in w nicht arbeiten können, müssen wir zu einem anderen Werk $u \in F_i \setminus \{w\}$ ausweichen. Auch hier reduziert sich das Problem den kürzesten Pfad zu finden. Wir wählen das u , dass die Distanz zu w plus u restliche Distanz zum Ziel (modes[0](u)) minimiert, da ab hier kein Streik mehr passieren kann.

Beweis für modes[1] (Streik noch möglich): Wir befinden uns in w und haben noch keinen Streik erlebt. Wir wollen uns zum nächsten Unternehmen bewegen und wählen dazu ein Ziel $u \in F_i$. Nachdem wir uns für u entschieden haben, ist der Gegner am Zug. Er hat zwei Optionen, um unseren Weg maximal zu verlängern:

- Option A: Er bestreikt genau das Werk u . Dann müssen wir den Ausweichplan ab u nutzen, was uns modes[2](u) kostet.
- Option B: Er spart sich den Streik für einen späteren Zeitpunkt auf. Dann wird der Streik zukünftig noch passieren modes[1](u).

Da wir das Worst-Case-Szenario betrachten, wird der Gegner stets das Maximum dieser beiden Optionen wählen: $\max(\text{modes}[1](u), \text{modes}[2](u))$. Da wir diese gegnerische Entscheidung antizipieren, wählen wir genau das Werk u , dass diesen maximalen Schaden minimiert. Die rekursive Formel $\min_{u \in F_{i+1}} (\text{dist}(w, u) + \max(\text{modes}[1](u), \text{modes}[2](u)))$ deckt also exakt dieses Spielverhalten ab.

Fazit: Da alle drei Modi nachweislich die korrekten Worst-Case-Entscheidungen modellieren, enthält der Zustand modes[1](S) am Startpunkt S auf jeden Fall die minimale Pfadlänge unter Berücksichtigung eines optimal spielenden Streik. $\square$

1.4 Laufzeitanalyse

Laufzeit:

- Sei n die Größe von V und e die Anzahl der Kanten
- Für jedes Werk führen wir Dijkstra aus, um die Distanzen zu allen Werken des nächsten und des gleichen Unternehmens zu finden. So besteht unserer Laufzeit aus $2n$ Wiederholungen von Dijkstra, dass in $\mathcal{O}((n + e) \log n)$ läuft.
- Im formalen Worst-Case: $\mathcal{O}(n \cdot (n + e) \log n) = \mathcal{O}(n^2 + n \cdot e \log n)$.
- Aber durch frühe Terminierung des Dijkstra, sobald alle Werke des Zielunternehmens gefunden sind, reduziert sich die praktische Laufzeit erheblich

Platzkomplexität:

- Graph: $\mathcal{O}(n + e)$
- Dijkstra-Zustände: $\mathcal{O}(n)$
- Ergebnisstrukturen: $\mathcal{O}(n)$
- Insgesamt: $\mathcal{O}(n + e)$

1.5 Cache-Lokalität und Speicheroptimierung

Um die Cache-Performance zu optimieren, verwenden wir eine spezielle Speicherorganisation. Der Graph wird in einem **flachen Array** gespeichert, wobei die Werke eines Unternehmens zusammenhängend im Speicher liegen.

Speicherorganisation:

1. **firm_loc[i]**: Startindex der Werke von Unternehmen i im **arena**-Array
2. **firm_size[i]**: Anzahl der Werke von Unternehmen i
3. **arena[]**: Flaches Array aller Werke, organisiert nach Unternehmen
4. S und T werden jeweils Firmen mit nur einem Werk an Stelle 0 und $m+1$. Bei ihnen darf kein Streik passieren.

Vorteile dieser Organisation:

- **Cache-Lokalität:** Bei Dijkstra-Läufen müssen wir häufig auf alle Werke eines Unternehmens zugreifen. Durch die zusammenhängende Speicherung vermeidet dies Cache-Misses und nutzt die CPU-Prefetching-Mechanismen optimal.
- **Einfache Indexierung:** Es ist sehr einfach alle Werk eines Unternehmens zu finden. Wir müssen uns nicht für jedes Unternehmen in einer Liste die Positionen all ihrer Werke merken.

Verbindungsspeicherung:

Jedes Werk speichert seine Verbindungen in Arrays (**conn** und **dists**). Diese sind für die Dijkstra-Nachbarschaftsuntersuchungen notwendig.

1.6 Potentielle Optimierung

Mithilfe eines **Multi-Source-Dijkstra** könnte dieser Ansatz theoretisch weiter verbessert werden. Anstatt für jedes einzelne Werk zwei separate Dijkstra-Suchen für alle Werke der gleichen und der nächsten Firma zu starten, könnte man die Suchen bündeln und “umdrehen” und so auf theoretisch drei Dijkstra-Durchläufe pro Unternehmen reduzieren:

1. **Berechnung von `modes[0]` und `modes[1]`:** Um diese Werte für alle Werke $w \in F_i$ zu finden, initialisieren wir jeweils einen Multi-Source-Dijkstra. Die Startpunkte für die Suche sind alle Werke des Folgeunternehmens $u \in F_{i+1}$. Anstatt die Suchen mit einer Distanz von 0 zu starten, initialisieren wir diese Startpunkte direkt mit ihren bereits berechneten Kosten: `modes[0](u)` für den ersten Durchlauf und “`max(modes[1](u), modes[2](u))`” für den zweiten. Da der Dijkstra-Algorithmus garantiert, dass der erste gefundene Pfad zu einem Knoten auch der global kürzeste ist, müssen wir die Suche nur so lange laufen lassen, bis wir alle Werke $w \in F_i$ erreicht haben. Die dort ankommenden Distanzen entsprechen dann direkt den gesuchten minimalen Kosten für `modes[0](w)` und `modes[1](w)`.
2. **Berechnung von `modes[2]`:** Die Bestimmung von `modes[2](w)` ist mit einem Multi-Source-Dijkstra viel komplexer. Hier suchen wir für jedes Werk $w \in F_i$ den günstigsten Ausweichweg zu einem anderen Werk $u \in F_i$ desselben Unternehmens.
 - **Das Problem:** Würden wir einen normalen Multi-Source-Dijkstra mit allen Werken $u \in F_i$ (initialisiert mit `modes[0](u)`) starten, hätte jedes Werk w sich selbst schon direkt als nächstes Ziel (mit Distanz 0).
 - **Die Lösung:** Wir modifizieren den Dijkstra-Algorithmus so, dass er für jeden Knoten im Graphen nicht nur die beste, sondern auch die zweitbeste Distanz speichert. Wichtig ist, dass diese beide Wege zwingend von zwei unterschiedlichen Startpunkten aus F_i stammen müssen. Hierdurch können sich die Suchbereiche der verschiedenen Startpunkte überlagern. Wenn wir am Ende die Ergebnisse für ein Werk w betrachten und der beste Weg von w selbst stammt (was wir ja ausschließen wollen, da w in diesem Szenario streikt), nehmen wir einfach den gespeicherten zweitbesten Weg. Dieser stammt garantiert von einem anderen Werk $u \in F_i$ und liefert uns exakt die minimalen Kosten für `modes[2](w)`, durch die Eigenschaft von Dijkstra. Dieser modifizierte Dijkstra benötigt in der Praxis zwar etwa doppelt so viel Zeit wie ein normaler, hat aber die gleiche theoretische Laufzeit.

Laufzeitbetrachtung: Dieser Ansatz reduziert die durchschnittliche Laufzeit von $\mathcal{O}(n \cdot (n + e) \log n)$ auf $\mathcal{O}(m \cdot (n + e) \log n)$, da wir jetzt pro Firma (insgesamt m) nur noch drei Dijkstra-Suchen ausführen müssen. Aber da die Anzahl der Firmen m theoretisch proportional mit n wächst und im schlimmsten Fall $m = n$ gilt, bleibt die Worst-Case-Laufzeit theoretisch unverändert bei $\mathcal{O}(n \cdot (n + e) \log n)$. Aufgrund der hohen Implementierungskomplexität dieser Optimierung und der Tatsache, dass unser implementierter Single-Source-Ansatz durch die frühe Terminierung in der Praxis bereits extrem effizient arbeitet und auch große Instanzen in wenigen Sekunden löst, ist dieser Ansatz eher nur von theoretischem Interesse.

1.7 Datenstrukturen

Hauptstrukturen:

```
Factory :: struct {
    firm : int,                // Unternehmen-ID
    id : int,                  // Werk-ID (0-basiert angepasst)

    conn : [dynamic]^Factory,  // Nachbarwerke
    dists : [dynamic]int,      // Distanzen zu Nachbarn
    modes : [3]Mode,           // DP-Zustände

    // Temporäre Dijkstra-Variablen:
    add_dist : int,            // Aktuelle Distanz
    came_from : ^Factory,      // Vorgänger für Pfadrekonstruktion
    final_ers : int,           // Finalisierungs-Versionszähler
    dist_ers : int             // Distanz-Versionszähler
}

Mode :: struct {
    cost : int,                // Kosten des Modus
    path : []^Factory          // Pfad (rückwärts)
}

Problem :: struct {
    factories : []Factory,     // Alle Werke (flaches Array)
    firm_loc : []int,          // Startindizes pro Unternehmen
    firm_size : []int,         // Anzahl Werke pro Unternehmen
    allowed_length : int       // Maximale erlaubte Zeit
}
```

2 Umsetzung

Programmiersprache und Werkzeuge Das Programm wurde in **Odin** implementiert. Es wurden nur `core` Bibliotheken verwendet. (“core:container/priority_queue”, “core:slice”, “core:os”, “core:fnt”, “core:strings”, “core:strconv”)

2.1 Dijkstra-Algorithmus mit früher Terminierung

Der Algorithmus basiert auf dem Dijkstra Algorithmus um die Distanzen zu allen Werken einer Firma ausfindig zu machen und die drei `modes` zu berechnen. Wir können ihn dadurch meist frühzeitig terminieren. Sobald alle benötigten Werke des Zielunternehmens gefunden wurden, bricht der Algorithmus ab. Dies ist möglich, da wir nur die kürzesten Pfade zu den Werken des nächsten Unternehmens benötigen und uns der Weg zu anderen Werken momentan egal ist. Um nicht nach jedem Dijkstra-Durchgang den Status aller Werke zurücksetzen zu müssen, haben wir eine für jeden Dijkstra-Durchgang einzigartige **Version**, die uns sagt, ob ein Werk schon in dem jetzigen Durchlauf verwendet wurde. (**Quelle 2**)

Implementierungsdetails:

- Verwendung einer Prioritätswarteschlange (`Priority_Queue`)
- Temporäre Variablen in der `Factory`-Struktur:
 - `add_dist`: Aktuelle Distanz vom Startwerk
 - `came_from`: Zeiger auf das Vorgängerwerk für Pfadrekonstruktion
 - `final_vers`: Versionszähler, um bereits besuchte Werke zu erkennen
 - `dist_vers`: Versionszähler für Distanzaktualisierungen

2.2 Minimierungsphase und Pfadrekonstruktion

Nachdem Dijkstra alle relevanten Werke erreicht hat, wird das optimale Zielwerk durch die `minimizer`-Funktion bestimmt: (**Quelle 3**)

```
Cost :: #type proc(^Factory) -> int
minimizer :: proc(candi : []Factory, start : ^Factory, cost : Cost) -> Mode
```

Diese Funktion:

1. Iteriert über alle Kandidatenwerke
2. Wählt das Werk mit minimalen Kosten entsprechend der gegebenen Kostenfunktion
3. Rekonstruiert den Pfad rückwärts über die `came_from`-Zeiger
4. Gibt eine `Mode`-Struktur mit Kosten und Pfad zurück.

Am Ende der Problemlösung überprüfen wir ob $\text{modes}[1](S) \leq L_{\max}$ ist. Ist das der Fall, dann werden die gesamten Haupt- und Ersatzwege folgendermaßen ermittelt und als finale Lösung ausgegeben:

1. Wir erstellen den Hauptweg, indem wir von S ausgehend den gespeicherten Weg in `mode[1]` (Streik noch nicht passiert) rückwärts ausgeben, bis wir an dem nächsten Zwischenziel ankommen und uns den dort gespeicherten Weg besorgen. Wir wiederholen dies so lange bis wir bei T angekommen sind.
2. Danach gehen wir diesen Weg nocheinmal durch. Für jedes Zwischenziel das wir treffen, gehen wir den gespeicherten Weg in `mode[2]` (Streik passiert jetzt) entlang, wodurch wir einen Streik an dieser Stelle simulieren. Daraufhin folgen wir allen `mode[0]` (Streik schon passiert) Wegen wie in 1 und geben die begangenen Strecken aus.

3 Beispiele**3.1 Pflichtbeispiele (lieferung00 bis lieferung10)**

Die genaue Struktur der Wege und der Ausfallwege ist zu groß für dieses Dokument, lässt sich aber im `ausgaben` Verzeichnis wiederfinden. Die benötigten Ausgaben in der Dokumentation sehen folgendermaßen aus:

lieferung00.txt

MOEGLICH

15

13

S [1 1] 2 [4 2] T

13

1 [2 1] [4 2] T

15

4 T [5 2] T

lieferung01.txt

MOEGLICH

40

32

S [1 1] [4 2] [8 3] T

46

1 S [2 1] S 1 [4 2] [8 3] T

37

4 [5 2] [8 3] T

36

8 [6 3] T

lieferung02.txt

MOEGLICH

188

127

S [1 1] S [2 2] [4 3] 3 5 [6 4] 5 [7 5] 13 2 S 1 T

143

1 6 [5 1] [3 2] [4 3] 3 5 [6 4] 5 [7 5] 13 2 S 1 T

162

2 13 7 5 [3 2] [4 3] 3 5 [6 4] 5 [7 5] 13 2 S 1 T

191

4 3 5 6 12 [11 3] [12 4] 6 5 [7 5] 13 2 S 1 T

143

6 [14 4] 6 5 [7 5] 13 2 S 1 T

127

7 [13 5] 2 S 1 T

lieferung03.txt

UNMOEGLICH

lieferung04.txt

MOEGLICH

264

lieferung05.txt

MOEGLICH

930

lieferung06.txt

UNMOEGLICH

lieferung07.txt

MOEGLICH

675

lieferung08.txt

MOEGLICH

1333

lieferung09.txt

MOEGLICH

11

lieferung10.txt

MOEGLICH

1462

4 Quellcode

Quelle 1 (aufgabe3.odin) | Hauptalgorithmus

```
aufgabe3 :: proc(p : ^Problem) {  
  
    search_vers := 0  
  
    for i := len(p.firm_loc) - 1 ; i >= 0 ; i -= 1 {  
  
        // find positions of current firm  
        start := p.firm_loc[i]  
        end := start + p.firm_size[i]  
  
        start2 := end  
        end2 := len(p.factories)  
        next_size := 1  
  
        if i != len(p.firm_loc) - 1 {  
            end2 = p.firm_loc[i + 1] + p.firm_size[i + 1]  
            next_size = p.firm_size[i + 1]  
        }  
  
        for j in start..  
end {  
            fac := &p.factories[j]  
            search_vers += 1  
  
            dijkstra(  
                fac,  
                next_size,  
                1,  
                search_vers  
            )  
  
            fac.modes[0] = minimizer(p.factories[start2:end2], fac, has_cost)  
            fac.modes[1] = minimizer(p.factories[start2:end2], fac, will_cost)  
        }  
    }  
}
```



```

// The starting node needs no now strike calculation.
if i == 0 {continue}

for j in start..

```

Quelle2 (aufgabe3.odin) | Dijkstra-Implementierung

```

dijkstra :: proc(start : ^Factory, num, mode, vers : int) {
    found := 0

    // setup priority queue
    que : pq.Priority_Queue(^Factory)
    pq.init(
        &que,
        proc(a, b : ^Factory) -> bool {
            return a.add_dist < b.add_dist
        },
        pq.default_swap_proc(^Factory),
        allocator = context.temp_allocator
    )
    defer free_all(context.temp_allocator)

    // fill first queue
    start.add_dist = 0
    start.dist_vers = vers
    start.came_from = nil
    pq.push(&que, start)

    // dijkstra
    for pq.len(que) > 0 {
        v := pq.pop(&que)

        if v.final_vers == vers {
            continue
        }
        v.final_vers = vers

        if v.firm == start.firm + mode {
            found += 1
        }
    }
}

```

```

    // Early exit if found needed
    if found == num {break}

    for i in 0..

```

Quelle 3 (aufgabe3.odin) | Minimierungsfunktion

```

Cost :: #type proc(^Factory) -> int
has_cost :: proc(f : ^Factory) -> int {
    return f.add_dist + f.modes[0].cost
}
will_cost :: proc(f : ^Factory) -> int {
    return f.add_dist + max(f.modes[1].cost, f.modes[2].cost)
}
now_cost :: proc(f : ^Factory) -> int {
    if f.came_from == nil {
        return max(int)
    }
    return f.add_dist + f.modes[0].cost
}

minimizer :: proc(candi : []Factory, start : ^Factory, cost : Cost) -> Mode {
    min := max(int)
    loc : ^Factory = nil

    for i in 0..

```

```
        os.exit(1)
    }

    route := make([dynamic]^Factory)
    for true {
        append(&route, loc)

        loc = loc.came_from
        if loc == nil {break}
    }

    return Mode{min, route[:]}
```